1	2	3	4	$\sum$

Freie Universität Berlin

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 08

Release: 04.12.2018; Due: 14.12.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 Binary Search Trees (__/10)

(a) *Insert the keys A, L, P, D, R, E, I, X, Y, Z in this order into a binary search tree that is initially empty. Next, delete the keys Z, A, L. Draw the tree after each insertion and after each deletion.*

Aufgabe im Anhang.

(b) *Let T_1 and T_2 be two binary search trees that both store the same set S of n entries. Show that there is a sequence of at most $2n$ rotations that converts T_1 into T_2.*

Wir haben uns folgenden Pseudocode überlegt.

```
convert(T1, T2):
    r1 = r(T1)
    r2 = r(T2)
    if r1 == null || r2 == null:
        return
    if r1 != r2:
        hoehe_pos = searchPos(r2, T1)
        rotate hoehe_pos mal
    convert(l(T1), l(T2))
    convert(r(T1), r(T2))
```

Listing 1: Pseudocode convert Tree

Wir möchten T1 in T2 umwandeln. Dabei suchen wir die Wurzel von T2 in T1 und geben uns die Höhe zurück. Um dieses Element zu unserer Wurzel zu machen, benötigen wir so viele Rotationen wie wir die Höhe haben. Unser Rekusionsanker ist, wenn wir unten angekommen sind und dann null als Knoten haben. Dann rufen wir rekursiv die Funktion mit den linken und den rechten Teilbäumen auf, bis T1 zu T2 umgewandelt wurde.

Im ersten Durchlauf haben wir im Baum n Knoten und somit haben wir eine Maximale Höhe von n. Somit muss im worst-case n Rotationen durchgeführt werden.

Im zweiten Durchlauf haben wir $\frac{n-1}{2}$ Knoten und somit im worst-case $\frac{n-1}{2}$ Rotationen.

Im zweiten Durchlauf haben wir $\frac{n-2}{4}$ Knoten und somit im worst-case $\frac{n-2}{4}$ Rotationen.

Damit können wir folgende Summe aufstellen:

$$\sum_{i=1}^{n} \frac{n-i}{2^i} \leq \sum_{i=1}^{n} \frac{n}{i^2} = n + \frac{n}{2} + \frac{n}{4} + \dots$$

Für große n können wir uns folgendes Bild überlgen:
Der erste Summand ist ein Quadrat. Der zweite Summand ist die Hälfte dieses Quadrates und wir zweichnen es daneben. Der dritte Summand ist wiederum davon die Hälfte, also ein virtel des Ursprünglichen Quadrates. Wenn wir das so weiterführen kommen wir immer näher an $2n$ heran, erreichen es nicht. Wenn wir n gegen Unendlich laufen lassen würden, ergibt die Summe $2n$. Da wir wissen, dass n begrenzt ist haben wir weniger als $2n$.

2 AVL-Trees (__/10)

(a) *Insert the keys A, L, P, D, R, E, I, F, O, M, N in this order into an AVL-tree that is initially empty. Next, delete A, M, R. Draw the tree after each insertion and after each deletion. Furthermore, show the individual rotations, and annotate the nodes of the trees with their heights.*

Aufgabe im Anhang.

(b) *Show that when inserting an element into an AVL-tree, we perform at most one (single or double) rotation. Does this also hold for a deletion? Justify your answer.*

Aufgabe im Anhang.

3 Weight-Balanced Trees (__/10)

Let T be a binary search tree. We define a weight function *w on T as follows: we set $w(\perp) = 1$. Let v be a node in T, with left subtree v_ℓ and right subtree v_r. Then, we set $w(v) = 1 + w(v_\ell) + w(v_r)$.*
We call T weight-balanced*, if for every node v in T, the following holds: let v_ℓ be the left and v_r be the right subtree of v. Then, we have $w(v_\ell) \geq w(v)/10$ and $w(v_r) \geq w(v)/10$.*
Show that any weight-balanced tree with n nodes has height $O(\log n)$.
Hint*: Use induction. Argue first that $w(v_\ell) \leq \frac{9}{10}w(v)$ and $w(v_r) \leq \frac{9}{10}w(v)$.*

Erstmal werden wir mit Induktion beweisen, dass $w(v_r) \leq \frac{9}{10}w(v)$ und $w(v_l) \leq \frac{9}{10}w(v)$

<u>I.A.:</u> $\rightarrow n = 1$
Damit wäre $w(v_r) = w(v_l) = 1$, weil die beiden leer sind.
Dann $w(v) = 1 + w(v_r) + w(v_l) = 3$
und dann überprüfen wir $w(v_r) = w(v_l) = 1 \leq \frac{27}{10} = 2,7$ - wahr

<u>I.V.:</u> gilt für n, also:
$w(v_l) \leq \frac{9}{10} \cdot w(v)$
$w(v_r) \leq \frac{9}{10} \cdot w(v)$

<u>I.S.:</u> $n \rightarrow n + 1$
Wir betrachten den zusätzlichen Gewicht des Baumes, wenn wir ein Knoten dazu addieren:

$w(v+1) = w(v) + 1$ - siehe bild
Man kann diesen neuen Knoten entweder im linken oder im rechten Teilbaum hinzufügen:

Fall 1.: Den neuen Knoten gehört zu linkem Teilbaum von v. Damit erhalten wir:
$w(v+1) = w(v) + 2$ und es muss gelten:
$\frac{9}{10} \cdot w(v+1) \geq w(v_r)$
$\frac{9}{10} \cdot w(v) + frac910 \geq w(v_r)$
Laut IV ist $\frac{9}{10} \cdot w(v) \geq w(v_r)$
Damit ist offensichtlich die Ungleichung richtig. Dann muss auch:
$\frac{9}{10} * w(v+1) \geq w(v_l + 1)$
$\frac{9}{10} \cdot w(v) + \frac{9}{10} \geq w(v_l) + 1$
$\frac{9}{10} \cdot w(v_l) + \frac{9}{10} \cdot w(v_r) + \frac{18}{10} \geq w(v_l) + 1$
$\frac{9}{10} \cdot w(v_r) + \frac{18}{10} \geq \frac{1}{10} \cdot w(v_l) + 1$
Wir wissen aus IV,dass es gilt $w(v_l) \leq \frac{9}{10} \cdot w(v)$
also am meisten den Wert $w(v_l) = \frac{9}{10} \cdot w(v)$ annehmen darf.
Aus dem Grund,dass der Baum gleichgewichtet ist,gilt es auch $w(v_r) \geq \frac{1}{10} \cdot w(v)$,also den niedrigsten Wert von wi$w(v_r) = \frac{1}{10} \cdot w(v)$
Wir ersetzen mit beiden,um zu sehen,ob die Gleichung stimmt, also den niedrigsten Wert des ersten Ausdrucks und die höchste des zweiten:
$\frac{9}{100} \cdot w(v) + \frac{18}{10} \geq \frac{9}{100} \cdot w(v) + 1$
$1,8 \geq 1$ - wahr.

Fall 2. Den neuen Knoten gehört zu rechtem Teilbaum von v. Analog zu Fall 1
Dann werden wir einen zweiten Induktionsbeweis über die Höhe des Baums machen und zwar, wir wollen beweisen,dass $h(v) \leq c \cdot \log(n+1) \in O(\log(n))$
Induktionsanfang
v ist der einzige Knoten in dem Baum.Dann ist $h(v) = 0 \leq \log 1 + 1$ - wahr.
Induktionsvoraussetzung
$h(v_r) \leq \log n_r + 1$
$h(v_l) \leq \log n_l + 1$
Hier müssen wir nur etwas merken.Was man hier merken muss ist,dass sowohl $n_r + n_l + 1 = n$ als auch $w(n_r) + w(n_r) + 1 = n$
Der einzigste Unterschied ist , dass man die leeren Knoten,auch das leere Baum bei der Anzahl der Knoten als 0 zählt und bei dem Gewicht mit 1.
Außerdem ist $h(v) = 1 + h(v_r)$
Induktionsschritt: $n \to n+1$
$h(v) = h(v_r) + 1$
$\leq c \cdot \log(n_r + 1) + 1$ - I.V.
$\leq c \cdot \log((n+1) \cdot \frac{9}{10}) + 1$
laut dem vorigen Induktionsbeweis $= c \cdot \log(n+1) + c \cdot \log \frac{9}{10} + 1$
$= c \cdot \log(n+1) + \frac{\log \frac{9}{10}}{\log \frac{10}{9}} + 1$
$= c \cdot \log(n+1) - \frac{\log \frac{9}{10}}{\log \frac{9}{10}} + 1$
$= c \cdot \log(n+1)$ - bewiesen.